Movie Recommendation with Matrix Factorization

Overview

Learn latent user and item factors from sparse ratings and evaluate both rating error and held-out ranking.

Movie Recommendation with Matrix Factorization

Use case and experiment

Recommend unseen movies by learning patterns in user ratings. The dataset has 60 fictional users, 40 fictional movie IDs and 1500 unique observed ratings sampled from hidden three-dimensional user and item factors. IDs do not correspond to real movie titles. Ratings are continuous values clipped between one and five. This controlled dataset is designed to have a learnable low-rank structure.

Movie Recommendation with Matrix Factorization

Model mathematics

Estimate rating = global training mean + user bias + item bias + user-factor dot item-factor. Use three latent dimensions and stochastic gradient descent to minimize squared error with L2 regularization. Cache the user factor before updating both factor vectors so the paired gradients refer to the same pre-update state. Predictions are clipped to one through five. Latent dimensions capture statistical patterns and do not have

automatically interpretable genre meanings.

Movie Recommendation with Matrix Factorization

Evaluation and leakage prevention

Use 1000 training pairs, 250 validation pairs and 250 test pairs. Compare models trained for 40 versus 100 epochs using validation RMSE. Evaluate the selected model on test without retraining. Rating baseline is the global training mean. For ranking, exclude training and validation items for each user, then compare top five recommendations with test ratings at least 3.5. Macro recall averages only users with at least one relevant test

item. Compare with a training-item-mean ranking baseline.

Movie Recommendation with Matrix Factorization

Files, outputs and inference

project.py implements generation, factor learning, prediction and recommendation. results/model.npz stores user and item factors, biases and training mean. results/ratings.csv records generated user-item ratings; the first 1000 rows are train, the next 250 validation and the last 250 test. recommend(model,user,seen) excludes known items and returns item IDs plus predicted ratings. A separate catalog would be needed to display real

titles. The report includes user zero recommendations as an example.

Movie Recommendation with Matrix Factorization

Interview questions and extensions

Why factorization? It shares information across sparse user-item interactions using a small latent space. Why RMSE and recall? Rating accuracy and ranking usefulness measure different objectives. What is cold start? A new user or item has no learned factor. This implementation supports only known IDs and does not implement cold-start fallback. Random splitting is optimistic for future recommendations. Extend with a licensed public

ratings dataset, chronological splits, popularity and content fallbacks, diversity checks, group-aware coverage tests and implicit-feedback ranking objectives. Treat unobserved items as unknown rather than automatically disliked.

Movie Recommendation with Matrix Factorization

Measured results from this build

data: "60 fictional users, 40 fictional movies, 1500 unique observed ratings; 1000 train, 250 validation, 250 test"

selected_epochs: 100

selection: [{"epochs": 40, "validation_rmse": 0.3253645583912829}, {"epochs": 100, "validation_rmse": 0.22374833377333472}]

test_rmse: 0.2389550097712311

global_mean_baseline_rmse: 0.7286026278063412

macro_recall_at_5: 0.8183760683760684

item_mean_baseline_recall_at_5: 0.25

ranking_evaluated_users: 39

caveat: "Random warm-start split on synthetic data. Unobserved ratings are unknown relevance, and ranking evaluation is incomplete. No cold-start or temporal claims."

See results/metrics.json for the complete machine-readable output, including individual predictions where available.

Movie Recommendation with Matrix Factorization

Setup and repository map

Requires Python 3.10 or later and NumPy. Create a virtual environment with python -m venv .venv. On Windows activate it with .venv\Scripts\Activate.ps1, or use .venv\Scripts\python.exe directly if activation is restricted. Install dependencies using python -m pip install -r requirements.txt. Run python project.py to train and evaluate, then open results/report.html in a browser. Run python -m unittest discover -v for tests. common.py

handles shared metrics and reporting; project.py contains the model and experiment; test_project.py contains four checks. Results are generated locally. No paid API, GPU or external dataset download is required.

Movie Recommendation with Matrix Factorization

Reproducibility, GitHub and honest presentation

Random seeds make the experiment reproducible within normal numerical-library differences, but exact floating-point results can vary by NumPy build. Runtime requires numpy>=1.26,<3. The shipped metrics record the environment used for this build in BUILD_INFO.txt. A GitHub Actions workflow runs tests and the training experiment on Python 3.11. Keep API keys and private datasets out of Git. The MIT license permits reuse. This is an AI-

assisted educational starter: explain what you reviewed, modified and tested. Synthetic labels are deliberately generated from model-friendly processes, so favorable scores are not evidence of real-world generalization. Learn one project deeply and add a documented extension before presenting it as your work.